

STUDENT RECORD AND ACADEMIC MANAGEMENT SYSTEM

Data Organization Using Python Collections

Project Type	Python Programming Project
Core Concepts	Strings, Lists, Tuples, Sets, Dictionaries, Functions, Classes
Purpose	Manage student profiles, academic records, and class performance statistics.

Prepared as a project report with problem analysis, data-structure selection, algorithms, pseudocode, source code, and test cases.

1. Problem Analysis Report

Problem Definition

The objective is to design and implement a Student Record and Academic Management System using Python's built-in collection types. The program stores, organizes, updates, searches, and reports student information without requiring an external database library.

Inputs

- Personal details: roll number, registration number, name, date of birth, department, and email.
- Academic details: subject names, marks, and attendance percentage.
- Other details: clubs and menu/operation selections.

Outputs

- Student profile display and search results.
- Confirmation or error messages for add, update, and delete operations.
- Individual averages, class average, highest-scoring student, and department summary.

Scope and assumptions

This version keeps records in memory while the program runs. Data is not permanently saved after the program closes. Marks are assumed to be numeric and each student's subject list should correspond to their marks list.

2. Data Structure Selection Report

Structure	Implementation	Reason for Selection
Tuple	(Roll number, registration number, DOB)	Immutable grouping for key identity details; helps avoid accidental in-place changes.
String	Name, department, email, subject names	Stores text and supports formatting and string operations.
List	Subjects and marks	Maintains order and allows elements to be changed or appended.
Set	Departments and clubs	Stores unique values and supports union/intersection operations.
Dictionary	Roll number → student profile	Key-value storage provides average-case $O(1)$ lookup by roll number.

3. Algorithm Design Document

Algorithm 1: Add a New Student

1. Read the student's roll number and details.
2. Check whether the roll number already exists in the main dictionary.
3. If it exists, display a duplicate-roll-number error and stop.
4. Otherwise, create an identity tuple, subject and marks lists, and a set of clubs.
5. Store the completed student profile in the dictionary using the roll number as the key.
6. Add the department to the unique-departments set and display a success message.

Algorithm 2: Search a Student Record

1. Read the target roll number.
2. Look up that key in the student dictionary.
3. If found, display the student's personal and academic details.

4. Otherwise, display "Record Not Found".

Algorithm 3: Calculate Class Average and Find Top Scorer

1. If there are no records, display a warning and stop.
2. For each student, calculate average marks = sum of marks ÷ number of subjects.
3. Add each student's average to the class total.
4. Track the student with the highest individual average.
5. Divide the sum of student averages by the number of students.
6. Display each student's average, overall class average, and top scorer.

4. Pseudocode Document

```
START
    student_records = empty dictionary
    unique_departments = empty set

    FUNCTION add_student(roll, name, dept, dob, subjects, marks, clubs)
        IF roll IN student_records
            PRINT "Error: Roll Number already exists"
            RETURN
        ENDIF

        identity_tuple = (roll, dob)
        profile = {
            "Identity": identity_tuple,
            "Name": STRING(name),
            "Department": STRING(dept),
            "Subjects": LIST(subjects),
            "Marks": LIST(marks),
            "Clubs": SET(clubs)
        }

        student_records[roll] = profile
        unique_departments.ADD(dept)
        PRINT "Student added successfully"
    END FUNCTION

    FUNCTION compute_class_average()
        IF student_records is empty
            RETURN 0
        ENDIF

        total = 0
        count = 0
        FOR EACH student IN student_records.VALUES()
            IF student["Marks"] is not empty
                avg = SUM(student["Marks"]) / LENGTH(student["Marks"])
                total = total + avg
                count = count + 1
            ENDIF
        ENDFOR

        IF count == 0
            RETURN 0
        ENDIF
        RETURN total / count
    END FUNCTION
END
```

5. Complete Source Code (Python)

The following source code implements adding, searching, updating, deleting, calculating individual averages, and generating a class performance report.

```
"""
Student Record and Academic Management System
Uses strings, lists, tuples, sets, and dictionaries.
"""
```

```

class StudentManagementSystem:
    def __init__(self):
        self.records = {}          # Roll number -> student profile
        self.departments = set()

    def add_student(self, roll_no, reg_no, dob, name, dept,
                    subjects, marks, attendance, email, clubs):
        if roll_no in self.records:
            print(f"[Error] Roll number {roll_no} already exists.")
            return False

        subjects = list(subjects)
        marks = [float(mark) for mark in marks]

        if len(subjects) != len(marks):
            print("[Error] Each subject must have one mark.")
            return False
        if not marks:
            print("[Error] Enter marks for at least one subject.")
            return False
        if any(mark < 0 or mark > 100 for mark in marks):
            print("[Error] Marks must be between 0 and 100.")
            return False
        if not 0 <= float(attendance) <= 100:
            print("[Error] Attendance must be between 0 and 100.")
            return False

        identity_info = (roll_no, reg_no, dob)
        student_profile = {
            "Identity": identity_info,
            "Name": str(name),
            "Department": str(dept),
            "Email": str(email),
            "Subjects": subjects,
            "Marks": marks,
            "Attendance": float(attendance),
            "Clubs": set(clubs)
        }

        self.records[roll_no] = student_profile
        self.departments.add(str(dept))
        print(f"[Success] Record for {name} added.")
        return True

    def search_student(self, roll_no):
        student = self.records.get(roll_no)
        if student is None:
            print(f"[Error] Record {roll_no} not found.")
            return None

        roll, reg, dob = student["Identity"]
        print("\n" + "=" * 45)
        print(f"STUDENT RECORD: {student['Name'].upper()}")
        print("=" * 45)
        print(f"Roll Number      : {roll}")
        print(f"Registration No.: {reg}")
        print(f>Date of Birth    : {dob}")
        print(f>Department      : {student['Department']}")
        print(f>Email          : {student['Email']}")
        print(f>Subjects       : {' , '.join(student['Subjects'])}")
        print(f>Marks          : {student['Marks']}")
        print(f>Attendance    : {student['Attendance']:.1f}%")
        clubs = ' , '.join(sorted(student['Clubs'])) or "None"
        print(f>Clubs        : {clubs}")
        print("=" * 45)
        return student

    def update_student(self, roll_no, new_marks=None,
                      new_attendance=None):
        if roll_no not in self.records:
            print(f"[Error] Record {roll_no} not found.")
            return False

        student = self.records[roll_no]
        if new_marks is not None:
            new_marks = [float(mark) for mark in new_marks]
            if len(new_marks) != len(student["Subjects"]):
                print("[Error] Marks count must match subject count.")
                return False

```

```

        if any(mark < 0 or mark > 100 for mark in new_marks):
            print("[Error] Marks must be between 0 and 100.")
            return False
        student["Marks"] = new_marks

    if new_attendance is not None:
        new_attendance = float(new_attendance)
        if not 0 <= new_attendance <= 100:
            print("[Error] Attendance must be between 0 and 100.")
            return False
        student["Attendance"] = new_attendance

    print(f"[Success] Record {roll_no} updated.")
    return True

def delete_student(self, roll_no):
    if roll_no not in self.records:
        print(f"[Error] Record {roll_no} not found.")
        return False
    deleted = self.records.pop(roll_no)
    print(f"[Success] Record for {deleted['Name']} deleted.")
    # Rebuild set so departments reflect current records.
    self.departments = {
        item["Department"] for item in self.records.values()
    }
    return True

def calculate_average(self, roll_no):
    student = self.records.get(roll_no)
    if student and student["Marks"]:
        return sum(student["Marks"]) / len(student["Marks"])
    return 0.0

def generate_class_report(self):
    if not self.records:
        print("[Warning] No records available.")
        return

    averages = []
    print("\n\n" + "#" * 55)
    print("CLASS ACADEMIC PERFORMANCE REPORT")
    print("#" * 55)

    for roll_no, profile in self.records.items():
        avg = self.calculate_average(roll_no)
        averages.append((avg, profile["Name"], roll_no))
        print(f"Roll: {roll_no} | {profile['Name']:<18} | Avg: {avg:.2f}%")

    class_average = sum(item[0] for item in averages) / len(averages)
    top_avg, top_name, top_roll = max(averages, key=lambda item: item[0])
    print("-" * 55)
    print(f"Registered Students : {len(self.records)}")
    print(f"Class Average : {class_average:.2f}%")
    print(f"Top Scorer : {top_name} (Roll {top_roll}, {top_avg:.2f}%)")
    print(f"Departments : {' '.join(sorted(self.departments))}")
    print("#" * 55)

if __name__ == "__main__":
    app = StudentManagementSystem()

    app.add_student(
        roll_no=101, reg_no="REG202601", dob="2003-05-14",
        name="Alice Smith", dept="Computer Science",
        subjects=["Python", "DSA", "Maths"], marks=[88, 92, 79],
        attendance=91.5, email="alice@univ.edu",
        clubs=["Coding Club", "Robotics"]
    )
    app.add_student(
        roll_no=102, reg_no="REG202602", dob="2002-11-20",
        name="Bob Jones", dept="Electrical",
        subjects=["Circuit Theory", "Maths", "Python"],
        marks=[75, 81, 85], attendance=84.0,
        email="bob@univ.edu", clubs=["Music Club"]
    )

    app.search_student(101)
    app.update_student(102, new_marks=[80, 85, 90],
                       new_attendance=88.0)

```

```
app.generate_class_report()
```

6. Test Cases and Verification Report

Test ID	Scenario	Input	Expected Outcome	Status
TC-01	Add valid student	Roll 103, Charlie	Record added successfully	PASS*
TC-02	Duplicate roll number	Roll 101 again	Duplicate error displayed	PASS*
TC-03	Search existing record	Roll 101	Alice's details displayed	PASS*
TC-04	Search missing record	Roll 999	Record-not-found error	PASS*
TC-05	Calculate average	88, 92, 79	Average = 86.33%	PASS*
TC-06	Delete existing record	Roll 102	Record removed	PASS*

**These are expected/manual verification outcomes based on the described operations; run the program to confirm them in your environment.*

7. Sample Output and Calculation

For Alice Smith, marks are 88, 92, and 79.

```
Average = (88 + 92 + 79) / 3
          = 259 / 3
          = 86.33%
```

For Bob Jones after the update, marks are 80, 85, and 90.

```
Average = (80 + 85 + 90) / 3
          = 255 / 3
          = 85.00%
```

```
Class average = (86.33 + 85.00) / 2
               = 85.67% (approximately)
Top scorer = Alice Smith (86.33%)
```

8. Conclusion

The project demonstrates how Python collections can be combined to manage student and academic information. A dictionary provides the main record index, tuples group identity information, lists hold ordered subjects and marks, sets track unique clubs and departments, and strings store text. The program supports core record operations and produces a simple academic summary.

9. How to Run the Program

1. Install Python 3 on your computer.
2. Copy the source code into a file named `student_management.py`.
3. Open a terminal in the folder containing the file.
4. Run: `python student_management.py`
5. The sample records will be added, one record will be displayed, Bob's marks and attendance will be updated, and a class report will be printed.